

Case Study (List Function)

List Function Prompt

Below are several examples of input and output lists.
There exist a function that maps the input list to the output list.

Input1: [81, 18, 0, 41, 0, 89] Output1: [41, 0, 89]
Input2: [43, 47, 41, 83, 0, 22] Output2: [22]
Input3: [20, 89, 0, 53, 73] Output3: [53, 73]

Now, based on the new input list, please perform reasoning and provide the output list.

Input: [18, 91, 89, 21, 0, 53, 40, 93]

Generate the correct output. Please provide the python function that turns the input list into the output list.
Note that the python function should input a list and return a list.

```
{
  "reasoning": "your reasoning process",
  "function": "only the python function here",
  "output": "your output list"
}
```

Ground Truth

Python Function:

```
def func(x):
    # elements after the first 0
    return x[x.index(0)+1:]
```

Answer: [53, 40, 93]

LLM Output (Llama-3.1-405B Instruct Turbo)

Reasoning: (incorrect)

The function appears to be filtering out the numbers in the input list that are greater than the number that follows it. It seems to be iterating through the list from left to right, and whenever it encounters a number that is greater than the next number, it includes the next number in the output list. If the next number is 0, it skips the 0 and continues to the next non-zero number.

Function: (execution output: [89, 21, 53, 40])

```
def filter_list(input_list):
    output_list = []
    i = 0
    while i < len(input_list) - 1:
        if input_list[i] > input_list[i + 1]:
            if input_list[i + 1] != 0:
                output_list.append(input_list[i + 1])
            else:
                j = i + 2
                while j < len(input_list) and input_list[j] == 0:
                    j += 1
                if j < len(input_list):
                    output_list.append(input_list[j])
                    i = j - 1
        i += 1
    return output_list
```

Output: (correct)

[53, 40, 93]

Case Study (MiniSCAN)

MiniSCAN Prompt

Here is a task:Your task is to convert an input sequence into an output sequence based on specific rules. Each word in the input sequence either corresponds to a direct transformation into a word in the output sequence, or a rule that defines how the output sequence should be structured.

Below are some examples

Input1: schwuen Output1: bleently
Input2: synieusk Output2: flauw
Input3: syneaum Output3: hyaently
Input4: throds Output4: gloiark
Input5: synieusk glaently Output5: flauw flauw flauw
Input6: schwuen glaently Output6: bleently bleently bleently
Input7: synieusk zoully syneaum Output7: flauw hyaently flauw
Input8: syneaum zoully schwuen Output8: hyaently bleently hyaently
Input9: synieusk thrioks syneaum Output9: hyaently flauw
Input10: schwuen thrioks synieusk Output10: flauw bleently
Input11: synieusk glaently thrioks syneaum Output11: hyaently flauw flauw flauw
Input12: syneaum thrioks schwuen zoully synieusk Output12: bleently flauw bleently hyaently
Input13: synieusk thrioks syneaum glaently Output13: hyaently hyaently hyaently flauw
Input14: syneaum zoully schwuen thrioks synieusk Output14: flauw hyaently bleently hyaently

Now, based on the new input sequence, please provide the output sequence.

Input: throds zoully throds thrioks throds glaently

Your output should very strictly follow the json dict format below. All your answers must be within the curly braces:

```
{
  "reasoning": "your reasoning steps",
  "Output": "your output sequence"
}
```

Ground Truth

Required Rules:

"throds": "gloiark",
"glaently": "Repeat the former three times",
"zoully": "Put the latter one between the two formers",
"thrioks": "Swap the former with the latter"

LLM Output (Qwen-2.5-72B Instruct Turbo)

Reasoning: (rule incorrect, execution output: "gloiark gloiark flauw gloiark gloiark gloiark")

From the examples provided, we can deduce the following transformations: 'schwuen' -> 'bleently', 'synieusk' -> 'flauw', 'syneaum' -> 'hyaently', 'throds' -> 'gloiark', 'thrioks' -> 'flauw' when it follows 'synieusk' or 'schwuen', and 'zoully' acts as a placeholder that does not change the word but affects the order or repetition. 'glaently' causes the previous word to repeat three times. Applying these rules to the input 'throds zoully throds thrioks throds glaently':

- 'throds' -> 'gloiark'
- 'zoully' does not change the word but affects the order
- 'throds' -> 'gloiark'
- 'thrioks' -> 'flauw' (as it follows 'throds')
- 'throds' -> 'gloiark'
- 'glaently' causes the previous word 'gloiark' to repeat three times.

Output: (correct)

"gloiark gloiark gloiark gloiark gloiark gloiark"

B.5 Pattern execution with in-context demonstrations

During pattern execution in the standard CoT setting, LLMs can not only deductively apply the inferred pattern to the test instance but also be implicitly influenced by the in-context demonstrations. However, these demonstrations are absent in the execution step of the "Inference & Execution" setting. To better demonstrate this impact, we conducted an additional experiment that incorporates in-context demonstrations into the pattern execution step. The experimental results from Table 5 and Table 6 show that performance significantly improves after incorporating in-context demonstrations into the deductive execution step, even though they are not directly required in the task setting. These findings serve as key evidence of CoT's reliance on implicit reasoning from demonstrations, in addition to explicit rationales.

Table 5: Experimental Results of "Inference & Execution" with in-context demonstrations provided in the pattern execution step, in dataset MiniSCAN.

Model	CoT	Inference & Execution	Inference & Execution (execution with demos)
GPT-4o-mini	4.10	1.35	2.81 (+1.46)
GPT-4o	20.60	11.24	16.97 (+5.73)
Llama-3.1-70b	19.80	8.89	14.49 (+5.60)
Llama-3.1-405b	32.00	16.74	29.23 (+12.49)
Qwen-2.5-7b	1.42	1.21	1.92 (+0.71)
Qwen-2.5-72b	22.00	13.91	16.32 (+2.41)
Average	16.65	8.89	13.62 (+4.73)

Table 6: Experimental Results of "Inference & Execution" with in-context demonstrations provided in the pattern execution step, in dataset List Function.

Model	CoT	Inference & Execution	Inference & Execution (execution with demos)
GPT-4o-mini	36.88	31.00	37.81 (+6.81)
GPT-4o	48.72	43.81	47.50 (+3.69)
Llama-3.1-70b	37.92	31.35	37.92 (+6.57)
Llama-3.1-405b	47.76	36.40	40.76 (+4.36)
Qwen-2.5-7b	29.36	17.27	19.68 (+2.41)
Qwen-2.5-72b	45.04	37.99	52.01 (+14.02)
Average	40.95	32.97	39.28 (+6.31)